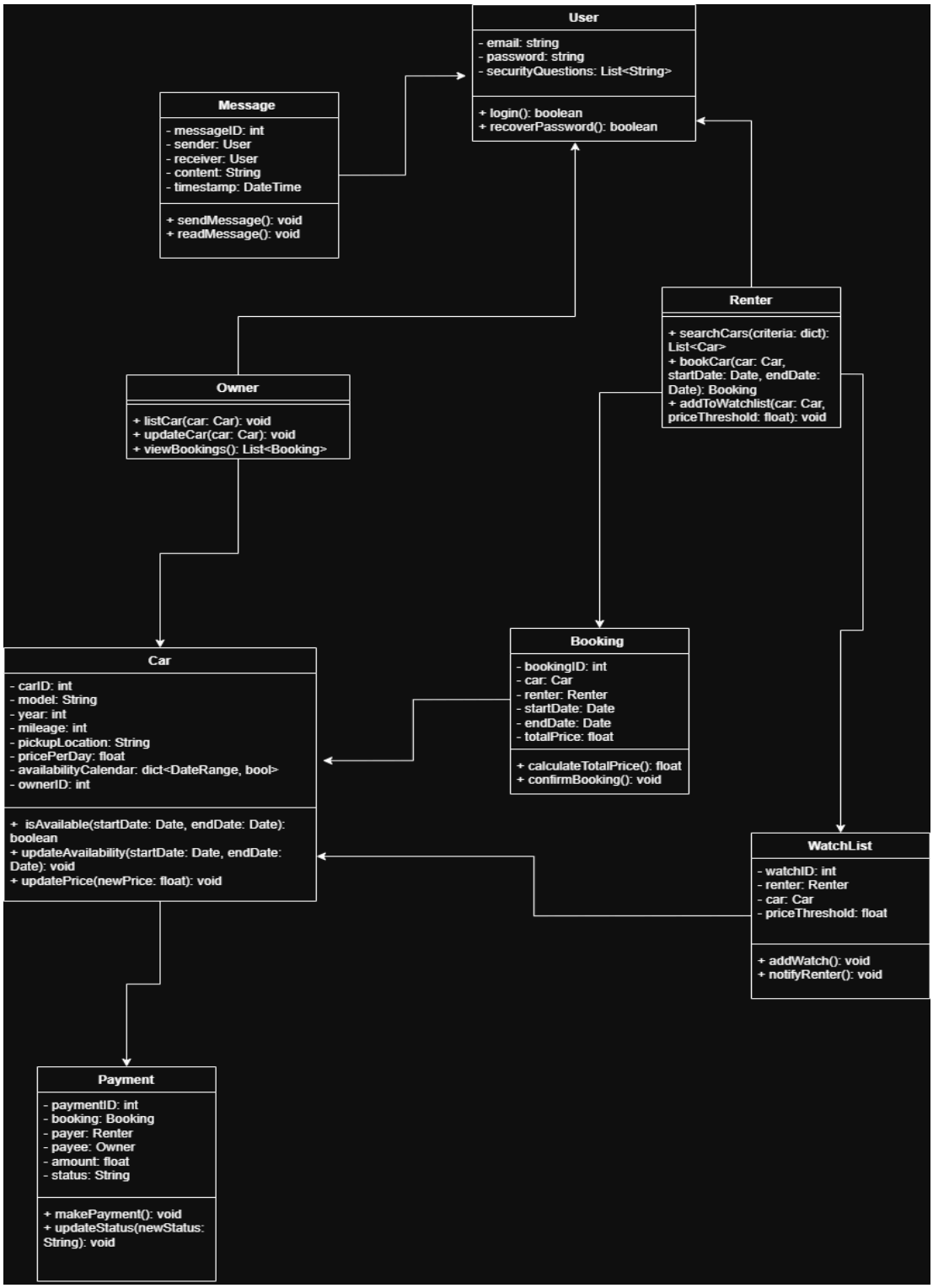
Zeinab Zeine

CIS 476

Term project report

Class Diagrams:

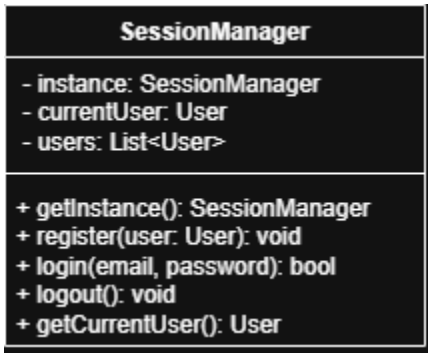


This full class diagram shows all the main classes used in DriveShare and how they relate to each other. The User class is at the top and acts as the parent class for both Owner and Renter. It holds the basic information that every user needs like their email, password, and security

question answers. Both Owner and Renter inherit from User, which means they automatically get those attributes plus their own.
The Owner class represents someone who wants to rent out their car. They can list cars, update car info, and view their bookings. The Renter class represents someone looking to rent a car. They can search for cars, make bookings, and add cars to their watchlist.
The Car class is one of the most important classes in the system. It stores all the details about a vehicle like the model, year, mileage, pickup location, price per day, and an availability calendar to track when it's booked. Cars are connected to owners, bookings, and the watchlist.
The Booking class is created when a renter reserves a car. It links a specific car and renter together with a start and end date, and automatically calculates the total price.
The Payment class handles the transaction for a booking. It keeps track of who paid, how much, and whether the payment went through.
The WatchList class lets a renter keep an eye on a car they're interested in. They can set a price threshold and get notified when the price drops to that amount.
Finally, the Message class handles communication between users on the platform, storing the sender, receiver, message content, and timestamp.

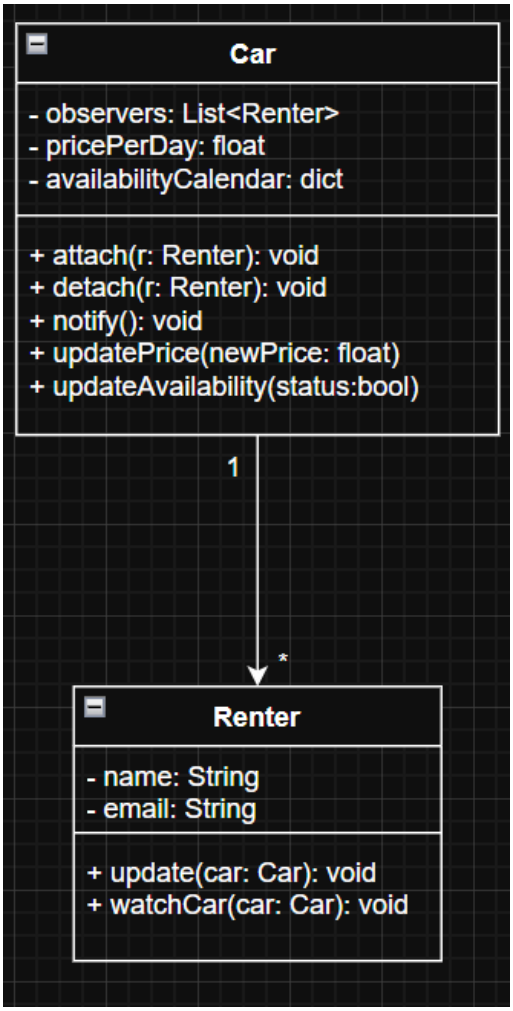
Singleton:

Uml:



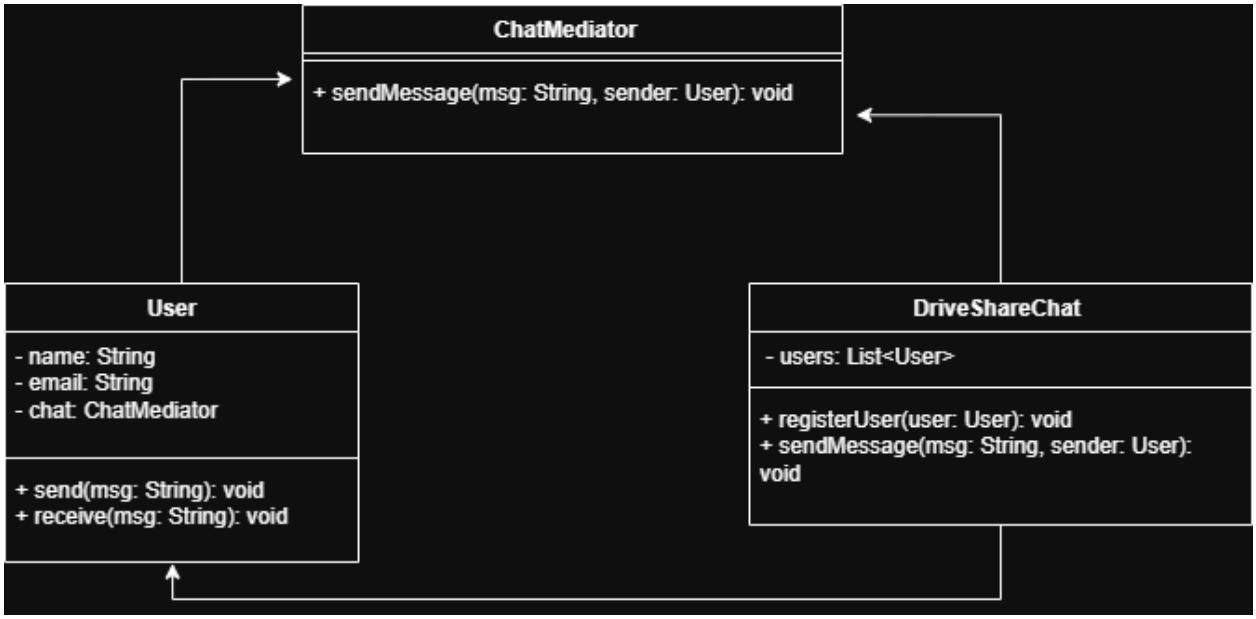
In DriveShare, I used the Singleton pattern to handle user authentication and session management. The idea behind this pattern is that only one session controller should exist at any time, so the system always knows exactly who is logged in. To do this, I created a SessionManager class that stores a single shared instance of itself, along with a simple in-memory list of users and a reference to the current user. Whenever someone tries to log in, the system calls SessionManager.getInstance() to access that one shared object. The login() method checks the email and password against the stored users, and if the information matches, it sets the currentUser to whoever logged in. There are also methods for logging out and checking who is currently active. Using the Singleton pattern here keeps everything consistent and prevents multiple sessions from being created accidentally. It also makes the authentication process easier to manage because all login-related logic is handled in one place instead of being spread across different parts of the program.

Observer:



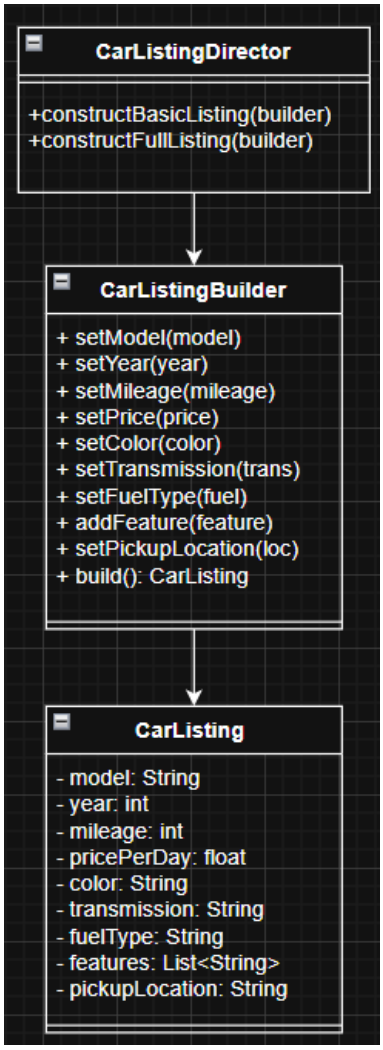
So, the Observer pattern is used to notify renters when a car they are watching becomes available or when the price drops. In this design, the Car class acts as the Subject, meaning it is the object being watched. Each car keeps a list of renters who are interested in it. Whenever something important changes, like the price or availability, the car calls its notify() method, which loops through all the renters in its observer list and alerts them.
The Renter class plays the role of the Observer. Renters can choose to “watch” a car by calling watchCar(), which internally attaches them to the car’s observer list. When the car updates its price or availability, the renter’s update() method is triggered, allowing the system to send a notification or message. This setup keeps the logic simple and clean: cars only need to know who is watching them, and renters only need to know how to react when something changes.

Mediator:



In DriveShare, the Mediator pattern is used to handle messaging between users. Instead of users sending messages directly to each other, they communicate through a central object called the DriveShareChat, which acts as the ConcreteMediator. This class stores a list of all registered users and is responsible for delivering messages to the correct recipients.
The ChatMediator is the abstract role that defines the method sendMessage(), which all mediators must implement. The DriveShareChat class implements this method by looping through all users and sending the message to everyone except the sender.
Each User acts as a Colleague in the pattern. A user has a reference to the mediator so they can call send(msg) without worrying about who receives it. When a message arrives, the mediator calls the user's receive(msg) method. This design keeps the communication logic centralized and prevents users from needing direct references to each other, making the system cleaner and easier to maintain.

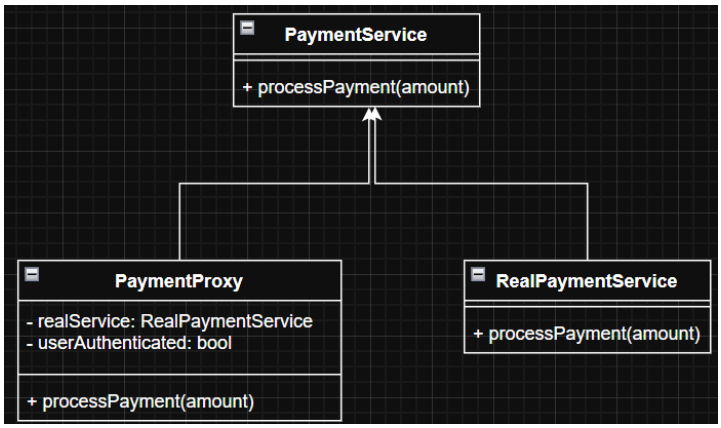
Builder:



The Builder pattern helps me create CarListing objects without dealing with a huge, messy constructor. A car listing can have a bunch of optional details (like color, features, pickup location ….), and instead of forcing all of that into one long parameter list, the Builder lets me set only the fields I actually need.
The CarListing class is the final product, it just stores all the information about a car someone wants to rent out. The CarListingBuilder is the class that actually builds this object step-by-step. Each setter method returns the builder itself, so I can chain the calls together in a clean, readable way. When I'm done setting everything, I call build() and get the finished CarListing.
The CarListingDirector is basically a helper that creates "preset" listings. For example, it can build a basic listing with only the required fields, or a full listing with all the optional details. The Director doesn't build the object itself, it just tells the Builder which steps to run and in what order. This keeps the code organized and avoids repeating the same setup code in multiple places.

We only have one builder because we're only building one type of object (CarListing). The variations come from which setter methods we choose to call, not from having multiple different ConcreteBuilders.

Proxy:

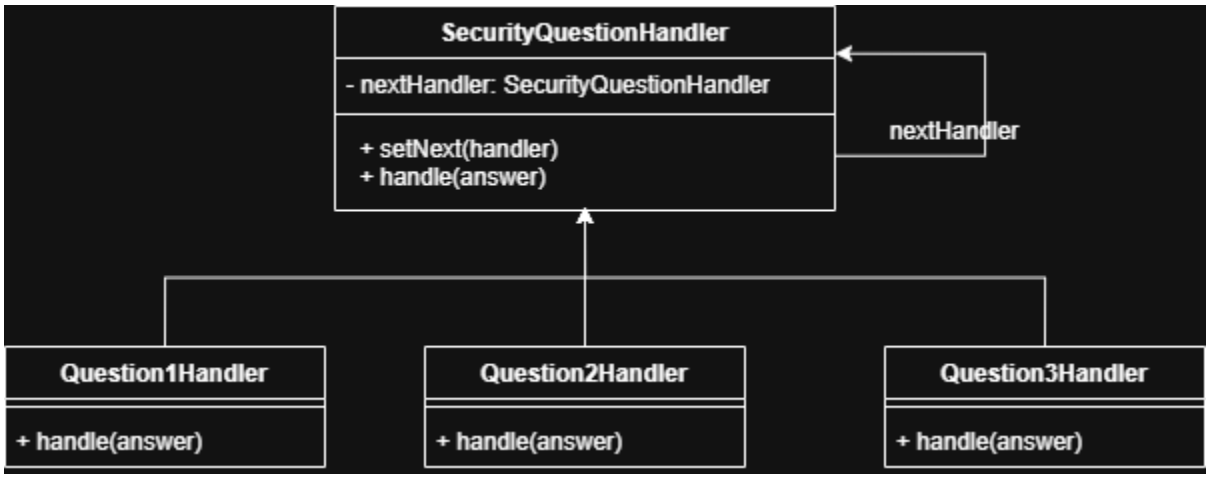


Now the Proxy pattern is used to control access to the real payment system. Instead of letting the app talk directly to the actual payment processor, I created a PaymentProxy that sits in the middle. The idea is that both the proxy and the real payment service share the same interface (PaymentService), so the rest of the program doesn't need to know which one it's using. They both implement the same processPayment(amount) method, which keeps everything consistent.

The RealPaymentService is the class that actually performs the payment logic. It represents the real system that would normally handle transactions. The PaymentProxy wraps around this real service and adds an authentication check before allowing the payment to go through. If the user isn't authenticated, the proxy blocks the request. If the user is authenticated, the proxy forwards the call to the real service.

This setup makes the payment process safer and more flexible. The UI only interacts with the proxy, not the real payment system, which means I can add security checks, logging, or other restrictions without changing the real payment code. The proxy basically acts like a gatekeeper, making sure only valid requests reach the real payment service.

Chain of Responsibility:



For password recovery in DriveShare, I used the Chain of Responsibility pattern to handle the three security questions that a user must answer before resetting their password. Each security question is represented as a separate handler class. The handlers are linked together in a chain, so if the user answers the first question correctly, the request moves on to the second handler, and then to the third. If the user gets any question wrong, the chain stops immediately and the recovery process fails.

This pattern works well because it lets me separate each question into its own class while still keeping the logic connected. It also makes the system easy to extend. For example, adding a fourth security question would just mean adding another handler and linking it to the chain. The final result is a clean, secure, and flexible password recovery process

Database Schema:

Since my DriveShare is a standalone console application, data is stored in-memory using Python objects rather than a relational database. These are the data models and their fields:

User

Field	Type	Description
email	String	Unique identifier for the user
password	String	User's login password
securityQuestions	List[String]	Three answers used for password recovery

Owner (extends User)

Field	Type	Description
cars	List[Car]	Cars owned and listed by this owner
bookings	List[Booking]	Bookings made on owner's cars

Renter (extends User)

Field	Type	Description
watchlist	List[WatchList]	Cars the renter is watching

Car

Field	Type	Description
carID	int	Unique car identifier
model	String	Car model name
year	int	Manufacturing year
mileage	int	Current mileage
pickupLocation	String	Where the car is picked up
pricePerDay	float	Daily rental price
availabilityCalendar	dict	Maps date ranges to booking status
ownerID	String	Email of the owner

Booking

Field	Type	Description
bookingID	int	Unique booking identifier
car	Car	The car being booked
renter	Renter	The renter who made the booking

startDate	date	Rental start date
endDate	date	Rental end date
totalPrice	float	Calculated total cost

Payment

Field	Type	Description
paymentID	int	Unique payment identifier
booking	Booking	The booking being paid for
payer	Renter	User making the payment
payee	Owner	User receiving the payment
amount	float	Payment amount
status	String	"Pending" or "Completed"

WatchList

Field	Type	Description
watchID	int	Unique watchlist entry identifier
renter	Renter	Renter watching the car
car	Car	Car being watched
priceThreshold	float	Price at which renter wants to be notified

Message

Field	Type	Description
messageID	int	Unique message identifier
sender	User	User sending the message

receiver	User	User receiving the message
content	String	Message text
timestamp	datetime	When the message was sent

Screen Shots:

OWNER log in:

```
=== Welcome to DriveShare ===

=== Login Menu ===
1. Login
2. Recover Password
3. Register
4. Exit
Choose an option: 1
Email: ali@google.com
Password: 1111

Welcome, ali@google.com!
```

Listing car:

```
=== Owner Menu ===
1. List a Car
2. Update Car Price
3. View Bookings
4. Send Message
5. Logout
Choose an option: 1

=== List a New Car (Builder Pattern) ===
Model: honda
Year: 2020
Mileage: 123456
Pickup Location: detroit
Price per day: 123
Car listed successfully: honda ($123.0/day)
```

Updating price:

```
=== Owner Menu ===
1. List a Car
2. Update Car Price
3. View Bookings
4. Send Message
5. Logout
Choose an option: 2

=== Update Car Price ===
1. Toyota Camry - $50/day
2. Honda Civic - $45/day
5. honda - $123.0/day
Enter Car ID to update: 5
Enter new price per day: 70
Price updated: honda is now $70.0/day
```

View booking:

```
Choose an option: 3

=== Your Bookings ===
1. Toyota Camry from 2026-12-12 to 2026-12-20 - renter: zeinab@google
.com - $400
```

Send message:

```
=== Owner Menu ===
1. List a Car
2. Update Car Price
3. View Bookings
4. Send Message
5. Logout
Choose an option: 4

=== Send Message ===
Users:
1. ali@google.com
2. maria@google.com
3. zeinab@google.com
4. john@google.com
Choose a user to message: 2
Enter your message: can we negotiate the price?
ali@google.com sends: can we negotiate the price?
maria@google.com received a message from ali@google.com: can we negot
iate the price?
```

Log out:

```
=== Owner Menu ===
1. List a Car
2. Update Car Price
3. View Bookings
4. Send Message
5. Logout
Choose an option: 5
Logging out...

=== Login Menu ===
1. Login
2. Recover Password
3. Register
4. Exit
Choose an option:
```

RENTER log in failed:

```
=== Welcome to DriveShare ===

=== Login Menu ===
1. Login
2. Recover Password
3. Register
4. Exit
Choose an option: 1
Email: zeinabzeine@google.com
Password: 2222
Invalid email or password.

=== Login Menu ===
1. Login
2. Recover Password
3. Register
4. Exit
Choose an option:
```

successful:

```
=== Login Menu ===
1. Login
2. Recover Password
3. Register
4. Exit
Choose an option: 1
Email: zeinab@google.com
Password: 1234

Welcome, zeinab@google.com!

=== Renter Menu ===
1. Search Cars
2. Book a Car
3. Make Payment
4. Send Message
5. View Watchlist
6. Add Car to Watchlist
7. Logout
Choose an option:
```

failed login menu:

```
=== Login Menu ===
1. Login
2. Recover Password
3. Register
4. Exit
Choose an option: 5
Invalid choice.
```

Search cars:

```
=== Renter Menu ===
1. Search Cars
2. Book a Car
3. Make Payment
4. Send Message
5. View Watchlist
6. Add Car to Watchlist
7. Logout
Choose an option: 1

=== Search Cars ===
Enter model or location keyword (or leave empty to see all): toyota

Search Results:
1. Toyota Camry (2020) - $50/day in Dearborn
```

Book a car:

```
76TermProject/main.py
3. Make Payment
4. Send Message
5. View Watchlist
6. Add Car to Watchlist
7. Logout
Choose an option: 2

=== Book a Car ===

Available Cars:
1. Toyota Camry (2020) - $50/day in Dearborn
2. Honda Civic (2019) - $45/day in Detroit
3. Tesla Model 3 (2021) - $90/day in Dearborn Heights
4. Ford Escape (2018) - $55/day in Ann Arbor
Enter Car ID to book: 3
Start year (e.g., 2025): 2026
Start month (1-12): 12
Start day (1-31): 1
End year: 2026
End month: 12
End day: 30

 Booking created!
Booking ID: 1
Car: Tesla Model 3
Total Price: $2610
```

Make a payment:

```
=== Renter Menu ===
1. Search Cars
2. Book a Car
3. Make Payment
4. Send Message
5. View Watchlist
6. Add Car to Watchlist
7. Logout
Choose an option: 3

=== Make Payment ===
Your bookings:
1. Tesla Model 3 from 2026-12-01 to 2026-12-30 - $2610
Enter Booking ID to pay for: 1
Proxy: User authenticated. Forwarding request to real payment service
...
Processing real payment of $2610...
Payment successful!
Payment recorded. Amount: $2610, Status: Pending
```

Add car to watchlist:

```
=== Renter Menu ===
1. Search Cars
2. Book a Car
3. Make Payment
4. Send Message
5. View Watchlist
6. Add Car to Watchlist
7. Logout
Choose an option: 6

=== Add to Watchlist ===

Available Cars:
1. Toyota Camry (2020) - $50/day in Dearborn
2. Honda Civic (2019) - $45/day in Detroit
3. Tesla Model 3 (2021) - $90/day in Dearborn Heights
4. Ford Escape (2018) - $55/day in Ann Arbor
Enter Car ID to watch: 4
Enter price threshold: 40
zeinab@google.com is watching Ford Escape at $40.0
```

View watchlist:

```
zeinab@google.com is watching Ford Escape at $40.0

=== Renter Menu ===
1. Search Cars
2. Book a Car
3. Make Payment
4. Send Message
5. View Watchlist
6. Add Car to Watchlist
7. Logout
Choose an option: 5

=== Your Watchlist ===
```

Register:

```
=== Login Menu ===
1. Login
2. Recover Password
3. Register
4. Exit
Choose an option: 3

=== Register ===
Enter email: zzeine@umich.edu
Enter password: 1234
Set your 3 security questions:
Answer to Q1 (What is your favorite color?): pink
Answer to Q2 (What city were you born in?): dearborn
Answer to Q3 (What is your mom's name?): hala
Are you registering as an (1) Owner or (2) Renter?
Choose 1 or 2: 1

Account created successfully! You can now log in as zzeine@umich.edu.

=== Login Menu ===
1. Login
2. Recover Password
```

Recover password:

```
=== Login Menu ===
1. Login
2. Recover Password
3. Register
4. Exit
Choose an option: 2
Enter your email: zzeine@umich.edu

=== Password Recovery ===
Question 1: What is your favorite color? pink
Correct answer.
Question 2: What city were you born in? dearborn
Correct answer.
Question 3: What is your mom's name?hala
Correct answer.
Your password is: 1234

=== Login Menu ===
1. Login
2. Recover Password
```

Exit:

```
=== Login Menu ===
1. Login
2. Recover Password
3. Register
4. Exit
Choose an option: 4
Goodbye!
PS C:\Users\zeina\Downloads\476TermProject>
```